

Atelier Tensorflow

Contexte

Une entreprise dispose de données provenant de plusieurs bâtiments équipés de capteurs. Pour chaque observation, on dispose par exemple de la température moyenne, de l'humidité, du nombre d'occupants, de l'heure, du jour de la semaine, de la surface du bâtiment et de la consommation énergétique.

L'objectif de l'atelier est de construire avec TensorFlow/Keras un **réseau de neurones** capable de **prédire la consommation énergétique d'un bâtiment à partir de caractéristiques telles que température, humidité et nombre d'occupants**.

Partie 0 – mise en place de l'environnement

- 1) Structurer le projet comme suit :

```
atelier_tensorflow_iot/  
├── notebooks/  
│   └── atelier_tensorflow_iot.ipynb  
└── models/  
    └── modele_consommation.keras
```

- 2) Créer le notebook `atelier_tensorflow_iot.ipynb`
- 3) Installer et importer `tensorflow`, `matplotlib` et `numpy`

Partie 1 – Génération du dataset

- 1) Générer **aléatoirement** 1000 valeurs pour chacune des variables suivantes :
 - a) **temperature** : valeurs qui suivent une loi normale avec une moyenne de 25 °C et un écart-type de 4 °C.
 - b) **humidite** : valeurs réparties de façon uniforme entre 30 % et 80 %.
 - c) **occupants** : valeurs entières choisies entre 1 et 49 inclus.
- 2) Déterminer la variable **consommation** avec la formule suivante :
 - la consommation de base (0 °C, pas d'humidité et pièce vide) est de 50
 - chaque degré supplémentaire augmente la consommation de 5 unités
 - chaque pourcentage d'humidité en plus ajoute 1,5 unité à la consommation
 - chaque personne présente dans la pièce augmente la consommation de 4 unités
 - dans la vraie vie, une formule mathématique parfaite n'existe pas. On ajoute donc une petite variation aléatoire (moyenne de 0 et écart-type de 10) pour simuler des imprévus ou d'autres facteurs non mesurés.
- 3) Rassembler les variables (**temperature**, **humidite** et **occupants**) dans la matrice des caractéristiques (features) **x** de taille 1000x3 en convertissant éventuellement les données au format (**float32**) optimisé pour les calculs
- 4) Créer la cible (target) **y** qui contiendra la variable **consommation**, au format **float32**

Partie 2 – Découpage Train/Test

Diviser le dataset précédent (**x** et **y**) en deux ensembles distincts : un pour l'entraînement (**train**) et un pour le test (**test**). Avec les conditions suivantes : 20% des données serviront au test ; garantir la reproductibilité du découpage.

Partie 3 – Création du modèle

- 1) Construire un réseau de neurones qui utilise l'architecture séquentielle suivante :
 - a) couche 1 : 16 neurones avec `relu` comme fonction d'activation
 - b) couche 2 : 8 neurones avec `relu`
 - c) couche 3 : 1 seul neurone.
- 2) Afficher un résumé textuel de l'architecture du réseau de neurones.

Partie 4 – Compilation du modèle

Compiler le modèle avec une méthode pour ajuster les poids, une fonction permettant de mesurer l'erreur de prédiction et une métrique permettant de suivre l'erreur absolue moyenne.

Partie 5 – Entraînement du modèle

- 1) Entraîner le modèle avec 20 % des données d'entraînement utilisés pour la validation, 50 passages sur les données et 16 observations traitées à la fois. Sauvegarder, dans la variable `history`, l'historique de l'entraînement
- 2) Examiner le contenu de `history`
- 3) Se servir de l'historique pour visualiser (graphique) et interpréter l'apprentissage

Partie 6 – Evaluation

- 1) Evaluer le modèle sur les données de test pour récupérer la fonction de perte et la métrique cible
- 2) Interpréter les résultats obtenus

Partie 7 – Prédiction

- 1) Faire des prédictions avec les données de test
- 2) Comparer les prédictions avec les valeurs réelles
- 3) Visualiser et interpréter cette comparaison. Que devrait-on observer si le modèle était parfait ?

Partie 8 – Sauvegarde du modèle

Sauvegarder le modèle pour avoir `models/modele_consommation.keras`

Partie 9 – Chargement du modèle

- 1) Charger le modèle
- 2) Afficher un résumé textuel du modèle chargé
- 3) Effectuer une nouvelle prédiction

Partie 10 : Fonction d'inférence

- 1) Avec le modèle chargé, définir une fonction `predire_consommation()`
- 2) Tester la fonction

Partie 11 : Bonus

Proposer et réaliser toute fonctionnalité pertinente et utile qu'on pourrait rajouter à l'atelier.

Livrable attendu

Chaque apprenant devra fournir le dossier `atelier_tensorflow_iot`

Modalité de livraison

Chaque apprenant

- 1) pousse le livrable, **d'abord vide**, sur un dépôt public de son compte Github personnel
- 2) met à jour le livrable, au fur et à mesure, avec des messages de commit explicites